A Crash Course in Git

(This is hopefully very helpful)

Resources – find the best way to learn for you

- CCC GitHub repository
- Boot.dev free-to-read course
  - Alternatively, their YouTube channel and the course video, where you can see the correct answer to the exercises. The first 2 hrs 30 mins are all you need for the basics.
- Git documentation
- Learn Git in Y minutes

Git

- Git is the most widely used version control system.
- It's used to:
  - Keep a history of code changes.
  - Revert mistakes made in code.
  - Collaborate with other developers.
  - Make backups of code.
  - Etc.

If for some reason you don't have Git...

- Check if you have Git installed:
  - `$ git --version`
- Install via:
  - `$ sudo apt install git-all`

Git config

- You can change the configuration at both the global and repository level. Mostly you just use the global config.
- Check your name and email:
  - `$ git config --get user.name`
  - `$ git config --get user.email`
- Set them using the following commands:
  - `$ git config --add --global user.name "github_username_here"`
  - `$ git config --add --global user.email "email@example.com"`
- You can also set the default branch for new repositories ("main" is the GitHub default but you can use "master"):
  - `$ git config --global init.defaultBranch master`
- You can check your global Git configuration in your .gitconfig file:
  - `$ cat ~/.gitconfig`

Repositories

- A repo is typically created for each project you work on.
- It's a directory that contains a project and the .git directory which stores all of the internal tracking and versioning information.
- If you enter the directory that you want to store your project, you can create a new repo:
- `$ git init`
- You can use `ls -a` to see the hidden contents of the directory and find the .git directory.

Status

- There are several states that a file can be in inside a Git repository. For example:
  - Untracked - not being tracked by Git.
  - Staged - marked for inclusion in the next commit.
  - Committed - saved to the repo's history.
- The command `git status` shows the current state of your repo and what status each file has.

Staging

- To stage a file to be included in the commit, it needs to be added with the command:
  - `$ git add <path-to-file>`
- If you don't stage a file, it won't be included in the commit – only files you explicitly add will be committed.
- You can stage all changes in your repo at once:
  - `$ git add .`

Commit

- After staging, files need to be committed.
- A commit is a snapshot of the repo at a given point in time.
- They come with a message that describes the changes made.
- How to commit all staged files:
  - `$ git commit -m "your message here"`
- If you mess up a commit message, you can change it with the amend flag:
  - `$ git commit --amend -m "my message here"`

Log

- A Git repo is a list of commits, where each commit represents the full state of the repo at a given point in time.
- The git log command shows a history of the commits in a repo.
- You can see:
  - Who made a commit.
  - When the commit was made.
  - What was changed.
- Each commit has a unique identifier, the "commit hash", which is a long string of characters.
- For convenience, each commit or change in Git can be referred to by using the first 7 characters of its hash.

Log

- Running git log starts an interactive pager you can scroll through with the arrow keys, and exit with q.
- You can turn off the pager with the `--no-pager` flag.
- You can also limit the maximum number of commits with the `-n` flag.
- For example:
  - `$ git --no-pager log -n 10`

Log

- The `--decorate=<argument>` flag can be used with the arguments:
  - short (default).
  - full (full ref name, which is a pointer to a commit).
  - no (no decoration).
- You can also use the `--oneline` flag to show a more compact view of the log:
  - `$ git log --oneline`
- The `--graph` flag shows the commit history visually with connections between parent and child commits.
- The `--parents` flag shows the parents of each commit.

Locations

- Git can be configured from several locations. For more general to more specific:
  - System: /etc/gitconfig, configures Git for all users on the system.
  - Global: ~/.gitconfig, configures Git for all projects of a user.
  - Local: .git/config, configures Git for a specific project.
  - Worktree: .git/config.worktree, configures Git for part of a project.
- 90% of the time you use --global, rest of the time use --local for project-specific configurations.
- A configuration set in a more specific location will override the more general location.

Branches

- A Git branch allows you to keep track of different changes separately.
- If you want to make a change in a big project, you can create a branch to experiment with the change and merge it back when you're done (or delete it).
- Check which branch you're currently on:
  - `$ git branch`
- To rename a branch:
  - `$ git branch -m oldname newname`

Branches

- You can also create a new branch and switch to it in one command:
  - `$ git switch -c my_new_branch`
  - The -c flag tells Git to create a new branch if it doesn't already exist.
  - If you want to branch off a specific commit, add that commit hash at the end of the command.
- When creating a new branch, the current commit is the branch base.
- The git checkout command is the old way of switching.
- Delete a branch via:
  - `$ git branch -d feature_branch`

Merge

- Once you have finished your changes on your branch, you need to merge them back into the main branch.
- If you have two branches:

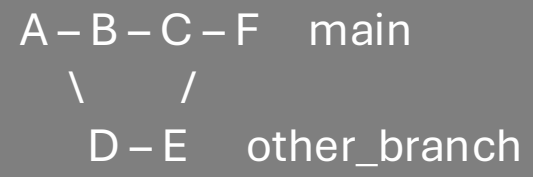
```
A – B – C    main
   \
    D – E    other_branch
```

- When merging other_branch into main, Git combines both branches with a new commit with both histories as parents, where F is a merge commit:

```
A – B – C – F    main
   \      /
    D – E    other_branch
```

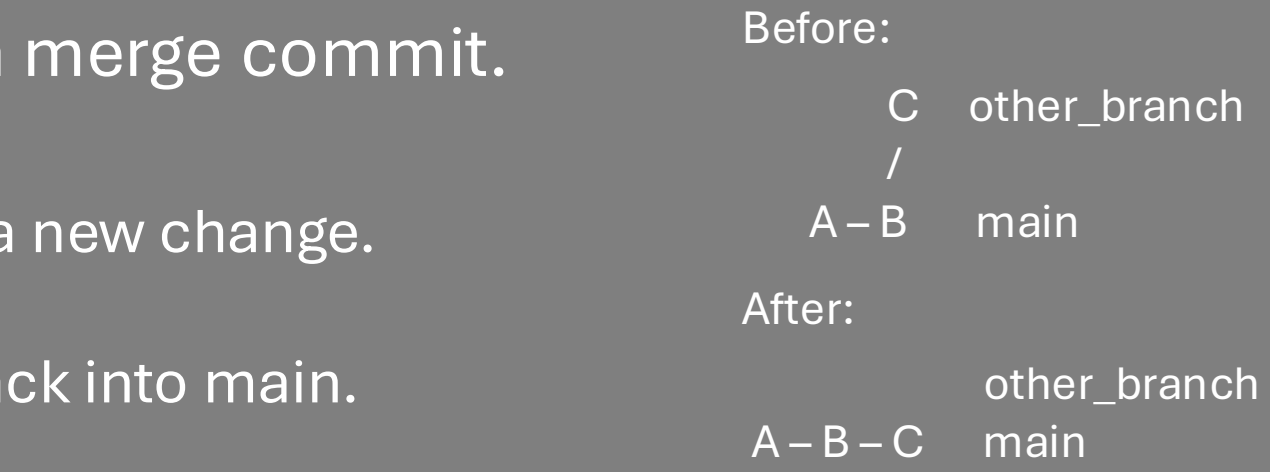
Merge

- To merge a branch into main, you need to switch to the main branch first – very important!
- Steps in a merge:
  - Find the "best common ancestor" (A).
  - Replay the changes from main, starting from A, into a new commit.
  - Replay the changes from the branch onto main, starting from A.
  - Record the results as a new commit, F.



Fast Forward Merge

- This words when a branch has all the commits that main has.
- Git will move the pointer of the "base" branch to the tip of the "feature" branch.
- This doesn't create a merge commit.
- A good workflow:
  - Create a branch for a new change.
  - Make the change.
  - Merge the branch back into main.
  - Remove the branch.
  - Repeat.



Rebase

- There is a lot of confusion between whether to use merge or rebase.
- If we have a commit history:

```
A – B – C     main
   \
    D – E     feature_branch
```

- Say we are working on the branch, and want to update the branch with changes from main.
- We could merge main into feature branch but that creates an additional merge commit.
- Rebase avoids a merge commit by replaying the commits from feature_branch on top of main.
- After a rebase, the history will look like this:

```
A – B – C           main
         \
          D – E     feature_branch
```

Rebase

- To use rebase to bring changes from main onto a branch, we run `$ git rebase main` while on the branch.
- This does the following:
  - Identifies latest commit from main to use as a temporary new base for the rebase process.
  - Replay each commit from the branch onto this location.
  - Update the branch to point to the last commit in the temporary location, so this is the new permanent branch.
  - The rebase doesn't affect the main branch and the other branch now includes all changes from main.

Merge vs Rebase

- Merge preserves the true history of a project.
- It shows when branches were merged and where.
- However, it creates a lot of merge commits, making the history harder to read and understand.
- Rebase provides a linear history that is easier to read and understand.
- Please don't ever rebase a public branch (like main) onto anything else – it could cause a lot of problems.

Reset

- git reset can be used to undo the last commit(s) or any changes staged or unstaged.
  - `$ git reset --soft COMMITHASH`
- The `--soft` option is useful if you want to go back to the previous commit but keep your changes.
- The committed changes are uncommitted and staged.
- If we don't want to keep changes:
  - `$ git reset --hard COMMITHASH`
- This is useful for discarding all changes and going bad to the previous commit
- Be careful with the `--hard` command, as you lose changes forever.
- If you want to reset to a specific commit:
  - `$ git reset --hard a1b2c3d`

Remote

- If someone else makes changes, you may need to accept them into your repo.
- We can have "remotes" which are external repos with mostly the same Git history as your local repo.
- For Git, GitHub is the remote, but it's just someone else's repo that we use for remotes.

Remote

- You treat the remote as the "truth" and often name it "origin". It contains the most up-to-date accepted version of the code.
  - `$ git remote add <name> <uri>`
  - For name, use origin. For the URI, the relative path to the remote.
  - You can have a remote locally too.

- You can use the git log command on the commits of a remote repo as well as a local repo.
  - `$ git log remote/branch`

Fetch

- To bring the contents of the remote to the local repo, we need to use the fetch command.
  - `$ git fetch`
- This copies objects and other information (metadata) from the remote into the current repo.
- This doesn’t copy the files and commits.
- You then need to merge the branches from origin onto your local repo:
  - `$ git merge remote/branch`
  - Run this inside the local repo, on the main branch.

GitHub

- This is the central website where "remotes" are hosted.
  - It's a backup of all your code.
  - A central place to share code and collaborate with others.
  - A public portfolio for your coding projects.
- Make sure you make a GitHub account.
- You can create "remote" repos hosted on GitHub servers.
  - Create an empty repo on GitHub.
  - Authenticate your local Git config with your GitHub account. Install the GitHub CLI: $ `curl -sS https://webi.sh/gh | sh`
  - Check you're authenticated: $ `gh auth login`

GitHub

- Add a remote to your local repo that points to the new GitHub repo:
- `$ git remote add origin https://github.com/your-username/your-repo.git`
- Run `$ git ls-remote` to make sure the remote was added correctly.

- To send changes to any remote, use the push command:
  - `$ git push origin main`
- This would push your local main branch commits to origin's main branch.

GitHub

- You can also push a local branch to a remote with a different name:
  - `$ git push origin <localbranch>:<remotebranch>`
- Delete a remote branch by pushing an empty branch to it:
  - `$ git push origin :<remotebranch>`

- To get the actual file changes from the remote locally:
  - `$ git pull [<remote>/<branch>]`
- The [...] means the remote and branch commands are optional.
- If you use `git pull` without any specification, it will pull your current branch from the remote repo.

Pull Requests

- A pull request is a way to propose changes to the maintainer of a project you're contributing to.
- Others can see what changes are being proposed and discuss before they are merged into the main codebase.
  - Stage and commit a change in a new branch and push it to GitHub.
  - On GitHub, click on the "Pull requests" tab and click "New pull request".
  - Set the base branch to main and the compare branch to your new branch.
  - Create the pull request.
  - Once approved, the merge button merges the changes into main.

Gitignore

- If using `git add .`, there may be some files you don't want to track.
- The .gitignore file at the root of your project solves this.
- Just add the directories/files that you don't want tracking.
- You can have multiple .gitignore files in the directories of a project.
- To ignore all .txt files, could use: `*.txt`
- Patterns starting with / are anchored to the directory containing the .gitignore file.
- For example, ignore a main.py file in the root directory, but not any subdirectories: `/main.py`

Gitignore

- Negate a pattern by prefixing with an exclamation mark. For example, to ignore all text files except one:
  - `*.txt`
  - `!important.txt`
- Add comments to the .gitignore:
  - `# Ignore all .txt files`
  - `*.txt`
- The order of patterns in the file matters, and they can override each other. For example:
  - `temp/*`
  - `!temp/instructions.md`

Gitignore

- Documentation
- What to ignore:
  - Things that can be generated (e.g. compiled code, etc.)
  - Dependencies (e.g. venv, pacjages, etc.)
  - Things specific to how you like to work or personal (e.g. editor settings)
  - Things that are sensitive or dangerous (e.g. .env files, passwords, API keys etc.)